

DESAFÍO TÉCNICO DE PRÁCTICA

Construcción de soluciones sobre Sparx Enterprise Architect y Prolaborate

Pasantías Proagile 2026



REGISTRO DE USO DE IA - EJERCICIO 1



Ezequiel Pino

Registro de Uso de **Inteligencia Artificial** - Ejercicio 1 - **Addino**

Introducción	3
Fase Inicial - Documentación	3
Evidencia	4
Fase de Desarrollo	6
Metodología SDD	6
Selección de Modelos	8
Registro Uso de IA	9
Prompts	12
DES-001:	12
DES-002:	13
DES-003	13
DES-004:	13
DES-005:	14
DES-006	15
Evidencia:	17

Introducción

Para la ejecución del desafío, utilice múltiples modelos, en distintas fases, según pruebas previas y benchmarks estudiados, para poder seleccionar el mejor modelo, para cada tarea. La idea era utilizar criterios eficientes, para poder exprimir al máximo el potencial de cada modelo. Las tareas de desarrollo tienen diferentes demandas cognitivas y estructurales, por eso, quizás implementar un modelo frontera, como Fable, nos traiga buenos resultados, pero no es efectivo económicamente, por los altos costos de consumo de créditos, o uso de API.

Para este desafío, mi objetivo fue utilizar los modelos de la forma más eficiente, cuidadosa y selectiva posible, ya que podemos obtener resultados al nivel de un modelo de frontera, pero cuidando la billetera. A continuación detallo el proceso en distintas fases

Aclaraciones IMPORTANTES: En la columna de “Estrategia-Prompt”, y en la columna de “Evidencia”, decidí colocar una referencia en la celda para aquellas peticiones largas. Utilizando la referencia, se puede buscar el prompt debajo del recuadro, es una decisión simplemente estética y de orden.

No se incluyen todos los prompts que se usaron, ya que si no el documento quedaría innecesariamente extenso. Solo se incluyen los prompts que inician las fases principales. Opte por colocar “Decisión Tomada”, en un cuadro aparte, por tema de disposición y prolijidad de presentación de la información.

Fase Inicial - Documentación

Para comenzar el desafío, lo más importante era comprender en profundidad, el entorno de Enterprise Architect, y la rigurosa y detallada documentación técnica que lo acompaña.

Para esto, cree un cuaderno de GeminiNotebook (Antes conocido como NotebookLM), para tener una fuente de conocimiento directa de toda la documentación técnica. Esto me ayudó a resolver dudas puntuales que surgen al momento de tomar decisiones de diseño importantes.

The screenshot displays the Sparx Enterprise Architect documentation interface. The main chat window shows a conversation about SQL queries for querying metadata in EA. The chat text includes:

el SQL Scratch Pad en el Database Builder) 3 4 y solo permiten sentencias SELECT; cualquier intento de modificación de datos (UPDATE o DELETE) es ignorado por seguridad s .

1. Tablas Clave del Repositorio de EA

Para construir cualquier consulta de gobierno o impacto, se trabaja principalmente con las siguientes tablas:

- `t_object`: Almacena las propiedades básicas de los elementos modelados (como su nombre, tipo, estereotipo, alias, estado, etc.) 6 - .
- `t_connector`: Registra los conectores o relaciones de todo tipo (dependencias, asociaciones, realizaciones) entre los elementos 6 - .
- `t_objectproperties`: Contiene los metadatos extendidos o Tagged Values asignados a los elementos 12 - .
- `t_package`: Define la jerarquía de paquetes que estructura el proyecto 6 - .

2. Cómo consultar Metadatos en EA con SQL

A. Consulta de Metadatos Básicos (Estado, Tipo, Estereotipo)

Muchos metadatos se encuentran directamente en las columnas de `t_object` (como `Object_Type`, `Stereotype`, `Status` y `Alias`) 7 13 .

Por ejemplo, si necesitas agrupar y contar tus aplicaciones (representadas típicamente como

The sidebar on the left lists various source documents, including "C_Sharp_Best_Coding_Practices.pdf", "Desafio_Tecnico_Practica_EA_Prolabor...", "enterprise_architect_sdk.pdf", "enterprise-architect-object-model.pdf", "Guia_Creacion_Addin_Enterprise_Archit...", and "InsideEA.pdf". The right sidebar shows options for generating content like "Resumen en audio", "Resumen en video", "Informes", "Cuestionario", "Tabla de datos", "Presentación con...", "Mapa mental", "Tarjetas didácticas", and "Infografía".

ID	Objetivo	Herramienta	Modelo	Estrategia -Prompt	Evidencia	Resultado
DOC-001	Cómo obtener solamente los elementos directos del paquete	Gemini Notebook	- Gemini 3.5 Flash	“Según el Object Model de EA, ¿qué colección contiene los elementos directamente incluidos en un Package? Confirmá si Package.Elements incluye subpaquetes o si estos se gestionan separadamente.”	Evidencia: DOC-001	Utilizar Package.Elements para obtener los elementos directos y tratar Package.Packages como una colección independiente.
DOC-002	Cómo validar que el usuario seleccionó realmente un paquete	Gemini Notebook	- Gemini 3.5 Flash	“Revisá en la documentación de EA cuál es la forma correcta de saber qué objeto está seleccionado en el Project Browser. Compará GetTreeSelectedPackage(), GetTreeSelectedItemType() y GetTreeSelectedObject(). Necesitamos aceptar únicamente una selección que sea realmente EA.Package.”	Evidencia: DOC-002	Se descartó usar directamente GetTreeSelectedPackage(), porque puede devolver el paquete contenedor de otro objeto seleccionado. Se decidió comprobar primero Repository.GetTreeSelectedItemType() == EA.ObjectType.otPackage y recién entonces recuperar GetTreeSelectedObject() y validarlo como EA.Package.
DOC-003	Qué hacer si Element.Update() devuelve false sin lanzar excepción	Gemini Notebook	- Gemini 3.5 Flash	“Investigá el contrato de Element.Update(). ¿Puede fallar devolviendo false sin lanzar una excepción? ¿Cómo deberíamos distinguir ese caso de una excepción?”	Evidencia: DOC-003	El flujo de guardado pudo distinguir false, excepciones y éxito por fila, continuando con los elementos restantes y conservando como pending las filas fallidas.

Evidencia

EV:DOC-001


Sparx Enterprise Architect documentation

+ Crear cuaderno
Copiar
Estadísticas
Compartir
Configuración
PRO

Fuentes

+ Agregar fuentes

Buscar fuentes nuevas en la Web

Web
Fast Research

Seleccionar todo

C_Sharp_Best_Coding_Practices.pdf
Desafio_Tecnico_Practica_EA_Prolabor...
enterprise_architect_sdk.pdf
enterprise_architect_sdk.pdf
enterprise-architect-object-model.pdf
Guia_Creacion_Addin_Enterprise_Archit...
InsideEA.pdf

Seleccionar todo

Seleccionar todo

Chat

Thoughts

La colección que contiene los elementos directamente incluidos en un paquete es `Package.Elements` 1 2 .

¿Se incluyen los subpaquetes en `Package.Elements` ?

No, la colección `Package.Elements` no incluye subpaquetes 2 3 . Los subpaquetes se gestionan de forma totalmente separada a través de la colección `Package.Packages` 3 .

Detalles de la Estructura en el Object Model:

- `Package.Elements` (Colección de objetos `Element`): Contiene únicamente los elementos de modelado de UML (como clases, casos de uso, componentes, etc.) que pertenecen directamente a la raíz de ese paquete 1 - . Para crear un nuevo elemento dentro de un paquete, se utiliza el método `Package.Elements.AddNew(Name, Type)` 5 6 .
- `Package.Packages` (Colección de objetos `Package`): Es una colección de lectura exclusiva que permite recorrer jerárquicamente todos los subpaquetes (paquetes secundarios) contenidos dentro del paquete actual 3 . Si un paquete tiene un `ParentID` de 0, significa que es un modelo raíz (sin paquete padre) 3 .
- El Atributo `Package.Element`: Cabe destacar que, aunque un paquete se maneja mediante la clase `Package`, cada paquete tiene una propiedad llamada `Element` (de tipo `Element`) que lo conecta con su representación correspondiente en la tabla de elementos (`Object`) 3 - 7 . Este objeto `Element` asociado permite consultar o modificar

Haz una pregunta o crea algo
7 fuentes

Studio

Resumen en audio
Resumen en video
Informes
Cuestionario
Tabla de datos

Presentación con...
Mapa mental
Tarjetas didácticas
Infografía

Los datos de salida de Studio se guardarán aquí. Luego de agregar fuentes, haz clic para agregar resumen de audio, una guía de estudio, un mapa mental más.

Agregar nota

EV:DOC-002


Sparx Enterprise Architect documentation

+ Crear cuaderno
Copiar
Estadísticas
Compartir
Configuración
PRO

Fuentes

+ Agregar fuentes

Buscar fuentes nuevas en la Web

Web
Fast Research

Seleccionar todo

C_Sharp_Best_Coding_Practices.pdf
Desafio_Tecnico_Practica_EA_Prolabor...
enterprise_architect_sdk.pdf
enterprise_architect_sdk.pdf
enterprise-architect-object-model.pdf
Guia_Creacion_Addin_Enterprise_Archit...
InsideEA.pdf

Seleccionar todo

Seleccionar todo

Chat

Thoughts

una selección que sea realmente `EA.Package`

Para identificar con total certeza el objeto seleccionado en el **Project Browser** de Enterprise Architect (EA) y asegurar que sea estrictamente un `EA.Package`, la documentación oficial establece diferencias críticas entre los tres métodos consultados 1 - :

1. Comparativa de Métodos del Object Model

- `GetTreeSelectedPackage()` : No sirve para validar la selección directa. Este método retorna el paquete en el cual el objeto seleccionado en el árbol está contenido 3 4 . Por ejemplo, si el usuario tiene seleccionada una Clase (un elemento de tipo `Element`) que pertenece al Paquete "X", `GetTreeSelectedPackage()` devolverá el objeto del Paquete "X" 3 - . Utilizarlo provocaría falsos positivos al procesar elementos comunes como si fueran el paquete seleccionado.
- `GetTreeSelectedItemType()` : Es la herramienta de filtrado y validación fundamental. Este método devuelve una enumeración de tipo `ObjectType` que identifica el tipo exacto del ítem seleccionado directamente en el árbol 2 - . Sus valores de retorno posibles para la navegación en el árbol son `otDiagram`, `otElement`, `otPackage`, `otAttribute` y `otMethod` 2 - . Al validar si el tipo devuelto es estrictamente `otPackage`, garantiza que el usuario ha seleccionado físicamente un paquete en la interfaz 1 6 .
- `GetTreeSelectedObject()` : Es el método para obtener el objeto real de manera

Haz una pregunta o crea algo
7 fuentes

Studio

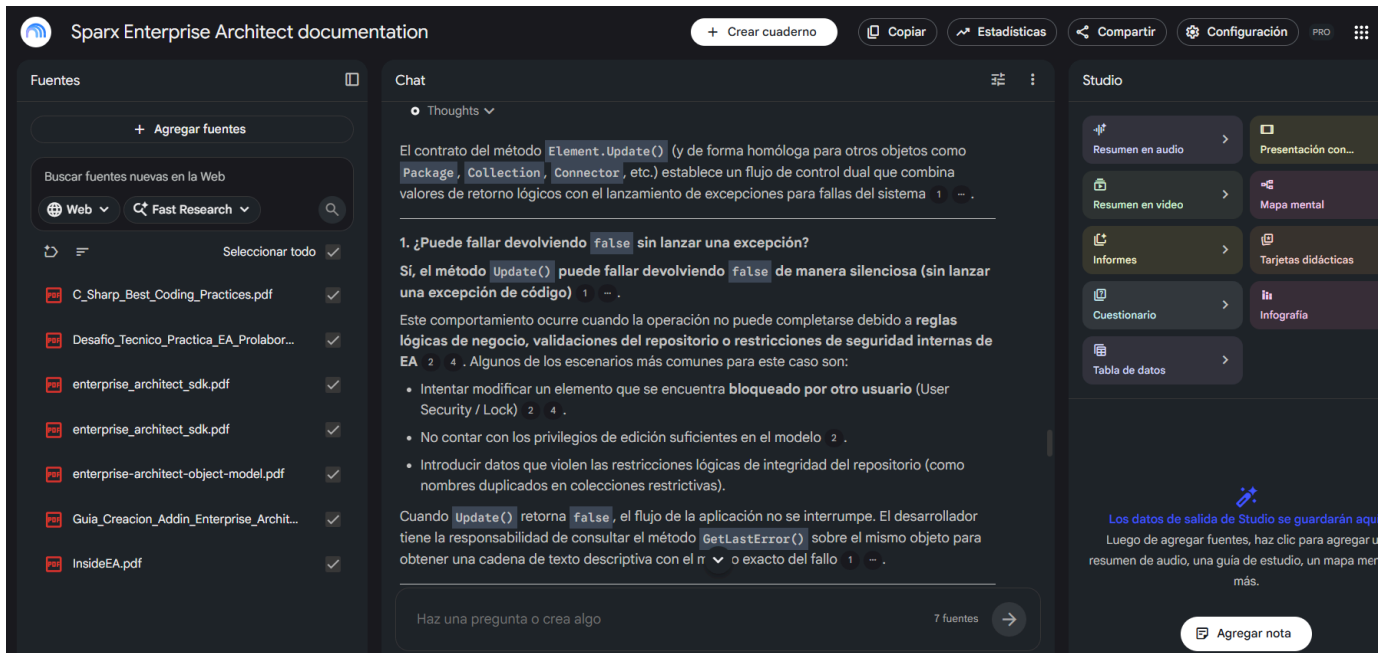
Resumen en audio
Resumen en video
Informes
Cuestionario
Tabla de datos

Presentación con...
Mapa mental
Tarjetas didácticas
Infografía

Los datos de salida de Studio se guardarán aquí. Luego de agregar fuentes, haz clic para agregar resumen de audio, una guía de estudio, un mapa mental más.

Agregar nota

EV:DOC-003



Fase de Desarrollo

Mi intención era usar modelos, de la forma más eficiente posible, con dos objetivos en mente, obtener resultados de calidad, y con el menor coste.

Para esto, adoptamos la suite de Gentle-AI, que implementa la metodología de Desarrollo Guiado por Especificaciones (SDD), detallada a continuación:

Metodología SDD

La metodología SDD estructura el ciclo de vida del desarrollo en fases lógicas y contratos formales e inmutables. Con esto, cada fase necesita los artefactos de las fases anteriores, facilitando la trazabilidad de decisiones de diseño e implementación. A continuación vemos un cuadro que resume cada fase:

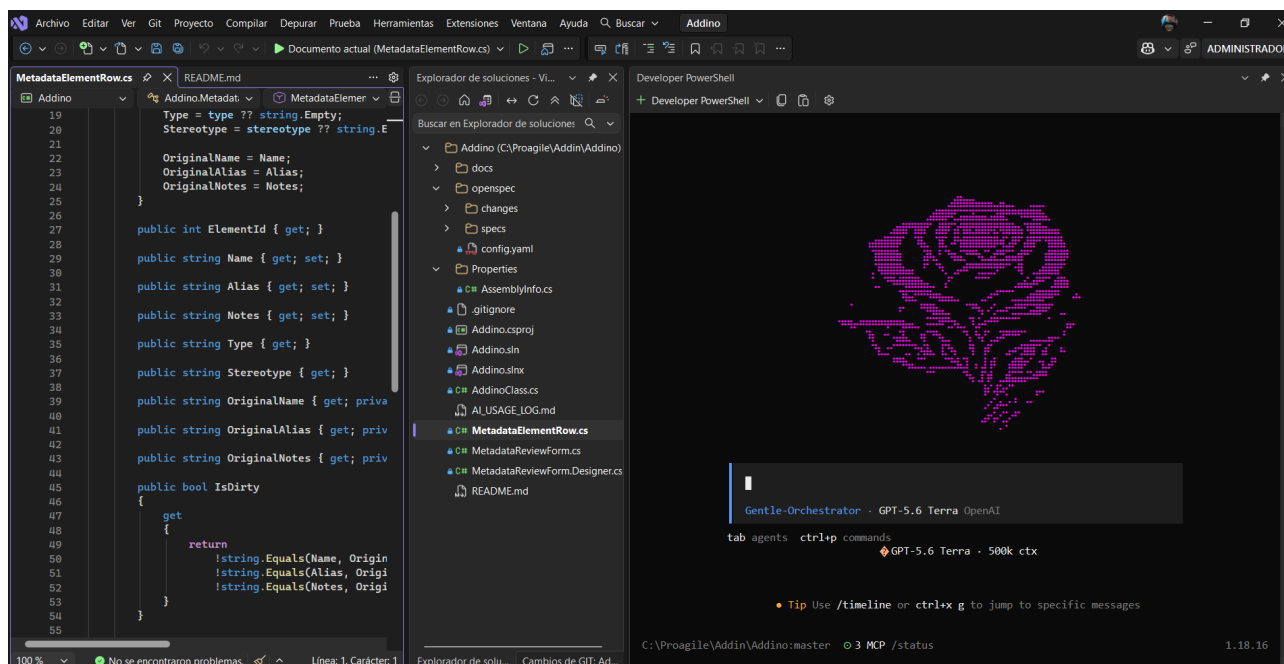


Fila del TUI	UI / Abstracción anterior	Función real
gentle-orchestrator	Coordinador (no es fase)	Agente coordinador del perfil: entiende la intención, decide cuándo entrar a SDD, delega a los subagentes de cada fase y sostiene el hilo de la sesión completa.
sdd-init	Inicialización	Detecta stack, scripts, convenciones, pruebas y estado mínimo del proyecto.
sdd-explore	Exploración	Investiga flujos, dependencias, archivos clave, riesgos y áreas impactadas.
sdd-propose	Propuesta / PRD operativo	Convierte la intención en alternativa recomendada, alcance, riesgos y criterios de decisión.
sdd-spec	Especificación OpenSpec	Formaliza requisitos, contratos, criterios de aceptación y comportamiento esperado.
sdd-design	Diseño técnico	Define arquitectura, cambios por capas, estrategia de datos, APIs, UI y pruebas.
sdd-tasks	Plan de tareas	Divide el diseño en pasos ejecutables, ordenados y verificables.
sdd-apply	Implementación	Aplica cambios en código, archivos, configuraciones, tests y scripts.
sdd-verify	Review / Audit	Valida diff, pruebas, riesgos, regresiones, seguridad y cumplimiento de spec.
sdd-archive	Cierre Commit-ready	Resume artefactos, deja bitácora, notas de commit y estado final.

También usamos un sistema de memoria persistente, bautizado como ENGRAM, que resuelve la “Amnesia operativa” que pueden tener los modelos entre secciones, o al momento de recibir múltiples peticiones seguidas. ENGRAM se sitúa entre el agente, y el sistema de archivos. Si el agente intenta leer o buscar un archivo, ENGRAM intercepta la llamada y le entrega un paquete contextual estructurado y reducido en lugar de un archivo crudo completo. A continuación dejo enlaces a repositorios de Github de este Ecosistema:

- Repositorio de Gentle-AI: <https://github.com/Gentleman-Programming/gentle-ai>
- Repositorio de ENGRAM: <https://github.com/Gentleman-Programming/engram>

De esta forma, convertimos autonomía cruda de modelos, en trabajo de ingeniería eficiente y controlado.



Utilizamos OpenCode en su forma TUI para desplegar Gentle AI en el entorno de Visual STUDIO, tal como se ve en la siguiente captura:

Se contempló utilizar la herramienta de VS Github Copilot integrada en Visual Studio, sin embargo lo descarté ya que podía controlar mejor el proceso de desarrollo utilizando Gentle-AI, utilizando SDD y control de versiones con GIT.

Selección de Modelos

Aca, asigne cada fase de SDD a distintos modelos, por tres razones fundamentales:

- Optimización de Costos de tokens
- Especialización de capacidades según tarea
- Preservación de la ventana de Contexto

Tenemos fases informativas, de síntesis, o de redacción, poner a un modelo potente, como Fable, o GPT SOL a realizar estas tareas no tiene sentido. Para esto realice una investigación donde encuentre el siguiente documento en la comunidad de Gentle-AI:

Perfiles SDD para Gentle-AI + OpenCode:

https://docs.google.com/document/d/15_b_dOgp6Wf5PI6RrZ6e0QGRBMXE2xnw/edit?usp=sharing&ouid=103625650383831231991&rtpof=true&sd=true

Donde muestran una combinación de los Modelos potentes de OpenAI, con los modelos que dispone la suscripción GO de opencode. En base a este documento, y con conocimiento de pruebas anteriores que he realizado, configure mi ecosistema de la siguiente forma en la TUI de opencode:

```
Assign Models to SDD, JD & Review Agents

LM Studio discovery failed; using configured models.

Current assignments:

gentle-orchestrator (coordinator) OpenAI / GPT-5.6 Terra
Set all SDD phases (not set)
▶ sdd-init OpenAI / GPT-5.6 Luna
sdd-explore OpenCode Go / MiniMax-M3
sdd-propose OpenAI / GPT-5.6 Terra
sdd-spec OpenCode Go / Kimi K2.7 Code
sdd-design OpenCode Go / MiniMax-M3
sdd-tasks OpenCode Go / Qwen3.7 Plus
sdd-apply OpenCode Go / Kimi K2.7 Code
sdd-verify OpenAI / GPT-5.6 Terra
sdd-archive OpenCode Go / MiMo V2.5
sdd-onboard OpenCode Go / MiniMax-M3
--- Judgment Day ---
jd-judge-a OpenAI / GPT-5.6 Terra
jd-judge-b OpenCode Go / Qwen3.7 Max
jd-fix-agent OpenCode Go / Kimi K2.7 Code
↓ more assignments

Continue
← Back
```


De esta forma, contamos con modelos potentes, en roles como orquestador, verificador, y delegamos al resto de modelos, con criterio, el resto de fases.

Al delegar el trabajo complejo a subagentes de fase independientes (los minions), se impide que la ventana de contexto del agente orquestador se contamine con un historial masivo de exploraciones, lecturas de archivos o comandos ejecutados.

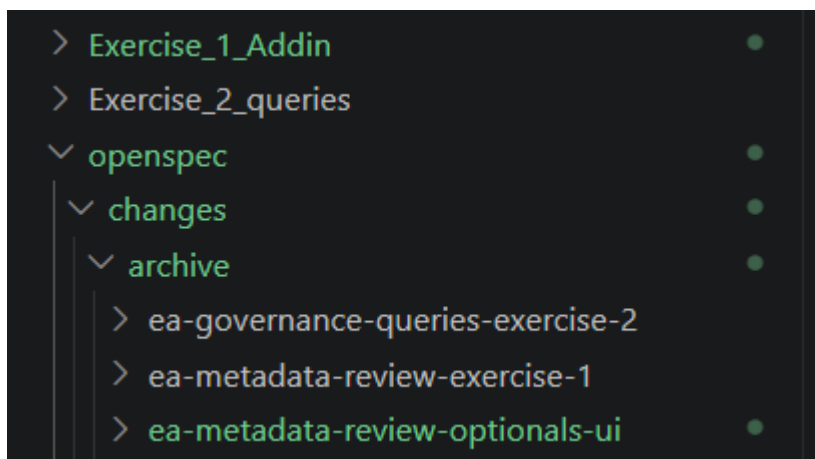
Cada subagente de fase inicia con una prompt y una ventana de contexto limpias, enfocándose estrictamente en su tarea sin arrastrar ruido, lo que asegura respuestas más estables y consistentes.

Durante el desarrollo del Ejercicio 1 se utilizaron distintos changes de OpenSpec para organizar el trabajo por etapas. Mientras una mejora se encontraba en desarrollo, su carpeta permanecía activa dentro de `openspec\changes\`, conteniendo artefactos como Proposal, Spec, Design, Tasks y Verify.

Una vez finalizada la implementación, aprobadas las validaciones correspondientes y completada la fase Archive, estos changes dejaron de estar activos y fueron trasladados a `openspec\changes\archive\` para conservar su trazabilidad histórica.

En el Ejercicio 1 se preservan dos changes principales:

- `ea-metadata-review-exercise-1`: contiene el desarrollo y validación del alcance base del Add-in.
- `ea-metadata-review-optionals-ui`: contiene la extensión posterior con los requisitos opcionales implementados, incluyendo validación de Name, carga recursiva, columna Paquete, indicador de cambios, Recargar y actualización visual.



Estas carpetas ya no representan trabajo pendiente, sino el registro final de decisiones, especificaciones, tareas y verificaciones realizadas durante el desarrollo.

Registro Uso de IA

A continuación, adjunto la tabla de registro de uso que usamos para algunas fases.

Cabe aclarar que en cada fase se usó GPT 5.6 TERRA como orquestador, delegando tareas a cada modelo. Cabe recalcar que los archivos que se mencionan a continuación, se encuentran en la ruta `\openspec\changes\archive`

ID	Objetivo	Herramienta	Modelo	Estrategia -Prompt	Evidencia	Resultado
DES-001	Inicializar formalmente el proyecto Addino dentro del flujo SDD y reconocer su estado técnico antes de planificar cambios Fase Init.	OPENCODE	GPT LUNA 5.6	Prompts: DES-001	Estado inicial del repositorio; Addino.csproj; AssemblyInfo.cs; configuración SDD/OpenSpec; commit baseline funcional.	Gentle AI reconoció correctamente el proyecto existente y dejó preparado el cambio para comenzar la exploración sin modificar la funcionalidad.
DES-002	Investigar el Ejercicio 1, el baseline Addino, requisitos, riesgos y decisiones que debían resolverse antes de proponer la solución. Fase Explore	OPENCODE	Minimax - M3	Prompts: DES-002	Creación de exploration.md; desafío técnico; guía del Add-in; inspección del baseline; decisiones humanas registradas al final del artefacto.	La exploración concluyó que no existían blockers técnicos para continuar y estableció una base verificada para Proposal.
DES-003	Definir formalmente alcance, intención, estrategia y límites del cambio sin entrar todavía en diseño detallado. Fase Propose	OPENCODE	GPT TERRA 5.6	Prompts: DES-003	Creación de proposal.md; exploration.md; Evidencia: DES-003	Se aprobó un alcance concreto y proporcional, con criterios de éxito y exclusiones claramente definidos.
DES-004	Convertir Proposal y requisitos del desafío en una especificación normativa y verificable para la implementación. Fase SPEC	OPENCODE	Kimi-2,7 Code	Prompts: DES-004	Creación de openspec/changes/ea-metadata-review-exercise-1/specs/ea-metadata-a-review/spec.md; Proposal; Exploration;	Se obtuvo una Spec implementable y verificable que convirtió la consigna en requisitos claros y escenarios de aceptación.

ID	Objetivo	Herramienta	Modelo	Estrategia -Prompt	Evidencia	Resultado
DES-005	Diseñar la arquitectura concreta de implementación respetando Proposal y Spec ya congelados.	OPENCODE	GPT 5.6 SOL	Prompts: DES-005	Creación de design.md; spec.md; proposal.md;	El diseño quedó aprobado sin preguntas abiertas y permitió pasar a Tasks con arquitectura, flujo de datos, manejo de errores y estrategia de validación definidos.
DES-006	Lanzar la etapa de Verify.	OPENCODE	GPT 5.6 SOL	Prompts: DES-006	Creacion de Verify-report.md	La verificación arrojó un resultado de tasks 18/18.

ID	Decisión tomada
DES-001	Mantener la infraestructura existente del Add-in como baseline y desarrollar el ejercicio sobre el mismo proyecto, sin alterar inicialmente su configuración COM ni plataforma.
DES-002	Reutilizar C#/ .NET Framework, WinForms, Interop.EA y el registro COM existente; validar estrictamente la selección mediante GetTreeSelectedItemType() antes de recuperar el EA.Package; mantener los opcionales fuera del alcance base.
DES-003	Implementar únicamente el alcance obligatorio: selección de paquete, elementos directos, edición local de Nombre/Alias/Notas, guardado explícito mediante Element.Update() y cancelación sin persistencia.
DES-004	Formalizar el comportamiento esperado mediante una Spec verificable antes de definir detalles de arquitectura o implementar código.
DES-005	Adoptar una arquitectura pequeña basada en AddinoClass, MetadataElementRow y MetadataReviewForm; desacoplar la UI de objetos COM persistentes y recuperar los elementos mediante ElementId únicamente al guardar.
DES-006	Someter la implementación completa a una verificación independiente contra consigna, Spec, Design, Tasks, código y evidencias antes de considerar cerrado el alcance obligatorio.

Prompts

DES-001:

Quiero comenzar el trabajo formal del desafío técnico de Proagile sobre el proyecto existente Addino. Para esta primera etapa realiza únicamente la fase de exploración SDD. No implementes código, no modifiques la funcionalidad existente y no avances a propuesta, especificación de diseño ni tareas.

CONTEXTO

- Addino es un Add-in para Sparx Enterprise Architect 17.1
- Está desarrollado sobre C# sobre .NET Framework 4.7.2
- Usa Interop.EA, COM, y plataforma X64
- Existe un commit Git baseline funcional anterior a la implementación del desafío.
- La funcionalidad actual Say Hello/Say Goodbye es únicamente código de prueba y será reemplazada posteriormente.

FUENTES DE VERDAD

Lee primero:

- @docs/challenge/Desafio_Tecnico_Practica_EA_Prolaborate_v2 (1).md
- @docs/challenge/Guia_Creacion_Addin_Enterprise_Architect.md

Luego inspecciona únicamente los archivos relevantes del proyecto actual. La consigna de @docs/challenge/Desafio_Tecnico_Practica_EA_Prolaborate_v2 (1).md tiene prioridad sobre CUALQUIER inferencia o recomendación.

Consulta source-material/pdf/ solo si aparece incertidumbre técnica que necesite validación.

No hagas una revisión general de todos los PDF de esta fase.

OBJETIVOS DE LA EXPLORACIÓN

Determina

1. Estado técnico actual de Addino y que infraestructura del Add-in de prueba puede reutilizarse.
2. Que partes de la funcionalidad actual deberán reemplazarse o ampliarse.
3. Los requisitos obligatorios, restricciones técnicas y criterios de aceptación del Ejercicio 1.
4. Los principales riesgos o incertidumbres que deban resolverse antes de rediseñar la solución, especialmente los relacionados son:
 - Selección del paquete en EA;
 - carga de elementos directos;
 - edición en memoria;
 - cancelación sin persistencia;
 - persistencia mediante Element.Update();
 - manejo de errores;
5. Separación explícita entre:
 - Requisitos obligatorios;
 - Mejoras de calidad posibles;
 - Desafíos opcionales;
6. Las validaciones que necesariamente deberemos realizar manualmente dentro de Enterprise Architect

RESTRICCIONES

- No implementes código

- No diseñes todavía una arquitectura definitiva.
- No agregues desafíos opcionales al alcance base.
- No cambies Framework, plataforma, COM ni tecnologías
- No trabajes sobre el ejercicio 2 de SQL/Prolaborate, salvo reconocerlo como un entregable separado (por ahora)
- No conviertas recomendaciones o supuestos en requisitos si la consigna no los exige.
- No tomes decisiones de diseño definitivas, si detectas alternativas, déjalas como decisiones abiertas para revisión humana.

Manten la exploración proporcional al tamaño actual del proyecto. Prioriza los archivos y requisitos directamente relevantes, y evita auditorías generales del entorno ya validado. Al finalizar, persiste el artefacto de exploración, presenta un resumen conciso con hechos verificados, riesgos e incógnitas, y detente para revisión humana antes de cualquier fase posterior.

DES-002:

Realizá únicamente `sdd-explore` sobre el proyecto Addino. Leé primero el desafío técnico y la guía de creación del Add-in. Determiná el estado actual, qué puede reutilizarse, qué debe reemplazarse, requisitos obligatorios, restricciones, criterios de aceptación, riesgos sobre selección de Package, elementos directos, edición local, cancelación y `Element.Update()`, y qué debe validarse manualmente en EA. Separá obligatorios, mejoras y opcionales. No implementes código ni diseñes todavía la arquitectura.”

Como complemento se incorporaron las decisiones sobre WinForms, español, `GetTreeSelectedItemType()`, `Package.Elements`, DTO local, Save explícito, errores por fila, `ShowDialog()`, `.sln` clásica y exclusión de opcionales.

DES-003

Ejecutá únicamente `sdd-propose` para `ea-metadata-review-exercise-1` usando la exploración y decisiones humanas aprobadas. Proponé el alcance mínimo necesario para cumplir el Ejercicio 1: acción de Extensions, validación estricta de `EA.Package`, grilla modal en español con elementos directos, edición mediante DTO local, persistencia explícita con `Update()`, `.sln`, README, evidencias y registro de IA. Mantené fuera del alcance los desafíos opcionales y el Ejercicio 2. No implementes código ni avances a Spec/Design

DES-004:

Ejecutá únicamente `sdd-spec` para el cambio aprobado. Convertí Proposal, Exploration y la consigna en requisitos técnicos verificables usando lenguaje MUST/SHALL y escenarios de aceptación. Especificá callbacks, validación de Package, carga de hijos directos, columnas y permisos, modalidad/idioma, ciclo local de edición, Save solo de filas modificadas, manejo de `Update() == false`/excepciones, plataforma y entregables. No diseñes clases o detalles de implementación que correspondan a Design y no incluyas opcionales.

DES-005:

Avanza Formalmente a `sdd-ddesing` para el cambio `ea-metadata-review-1`.

La Proposal y la Spec está probada y congeladas. No las modifique ni reabras decisiones funcionales ya resueltas. El checkpoint Git previo a Design ya fue realizado.

Usa como fuentes de verdad:

- Los documentos de `@docs/challenge\`
- `@openspec/specs/ea-metadata-review/spec.md`
- `@openspec/changes/ea-metadata-review-exercise-1/proposal.md`
- `@openspec/changes/ea-metadata-review-exercise-1/exploration.md`
- El código actual del proyecto Addino.

OBJETIVO:

Diseña cómo implementar el Ejercicio 1 obligatorio sobre el Add-in existente, de forma simple, robusta, mantenible y proporcional al tamaño del desafío. El diseño debe respetar sin cambios lo siguiente:

- C# sobre .NET Framework 4.7.2.
- Windows Forms.
- EA 17.1 x64 como target validado.
- Interop.EA y registro COM.
- Los cinco callbacks EA requeridos.
- Acción en Extensions.
- validación estricta de `EA.Package`;
- carga exclusiva de elementos directos;
- Name/Alias editables, Notes multilínea, Type/Stereotype read-only;
- edición local hasta Save;
- Cancel/Esc/X sin persistencia;
- `Element.Update()` solo para elementos modificados;
- procesamiento independiente y manejo de false, excepciones y elementos bloqueados;
- UI modal y visible en español.

Decisiones que debe resolver Design

Define y justifica la estructura concreta de implementación, incluyendo:

- componentes/clases necesarias y responsabilidad de cada una;
- flujo desde `EA_MenuClick` hasta la apertura del formulario;
- forma de representar localmente una fila y conservar una identidad - estable del elemento EA sin depender de Name/Alias;
- estrategia de `BindingList/binding` y detección interna de modificaciones;
- carga de datos desde `EA.Package.Elements`;
- configuración de `DataGridView`, incluyendo Notes multilínea;
- cómo asegurar que una celda todavía activa quede confirmada antes de Save;
- cómo recuperar de forma segura el `EA.Element` correspondiente durante la persistencia;
- flujo detallado de Save normal, Save sin cambios y Save parcial;
- cómo limpiar el estado pendiente de los éxitos y conservar pendiente el de los fallos;
- estrategia de mensajes/resumen de guardado en español;
- comportamiento de Cancel, Esc y X;
- manejo de excepciones COM y liberación/uso razonable de objetos COM;
- modificaciones previstas sobre `AddinoClass.cs`, nuevos archivos WinForms y `Addino.csproj`;
- eliminación completa de Say Hello y Say Goodbye;

- generación posterior de la solución clásica .sln;
 - tratamiento de Interop.EA para no romper la configuración actualmente funcional y permitir únicamente ajustes mínimos de portabilidad si fueran necesarios.
- Para cada decisión donde existan alternativas razonables, selecciona la opción más simple que satisfaga la Spec y explica brevemente por qué.
- Evita sobrearquitectura: no introduzcas interfaces, repositories, dependency injection, capas genéricas, patrones o abstracciones si no aportan una necesidad concreta a este Add-in pequeño. Tampoco acoples innecesariamente la UI directamente a objetos COM si existe una alternativa local sencilla y segura.
- El Repositorio Pasantías permanece externo al workspace Git y se utiliza únicamente para validación manual en Enterprise Architect.
- Mantén fuera del diseño: recursividad, creación de elementos, reload, dirty-row highlighting, validación obligatoria de Name vacío y todo el Ejercicio 2/Prolaborate.
- Considera los entregables únicamente en aquello que afecte al diseño o a la capacidad de ejecutar/verificar la solución; no generes ahora capturas ni video.

RESULTADO ESPERADO

Genera y persiste únicamente el artefacto correspondiente a `sdd-design`.

El documento debe dejar suficientemente claro:

- arquitectura/componentes;
- responsabilidades;
- flujo de datos;
- flujo de apertura;
- flujo de Save;
- manejo de errores;
- ciclo de vida del estado local;
- interacción con EA/COM;
- archivos previstos;
- decisiones técnicas y trade-offs;
- riesgos técnicos restantes y mitigaciones;
- trazabilidad con la Spec.

No implementes código, no generes tasks y no avances a `sdd-tasks` ni `sdd-apply`.

Al finalizar, resume las decisiones principales, señala únicamente blockers o riesgos reales que necesiten decisión humana y detente para revisión.

DES-006

Continúa formalmente con `sdd-verify` para el cambio `ea-metadata-review-exercise-1`.

La implementación y documentación del alcance obligatorio del Ejercicio 1 están completas y `tasks.md` se encuentra en 18/18.

Realiza una verificación rigurosa e independiente. No asumas que el cambio es correcto por estar marcado como completo.

Usa como fuentes de verdad, en este orden:

1. la consigna original del desafío técnico;

2. Spec;
3. Proposal y Design;
4. Tasks y sus notas de validación;
5. implementación actual;
6. README, AI Usage Log y evidencias disponibles;
7. historial/diff Git cuando sea útil para comprobar trazabilidad.

Verifica el cumplimiento del **alcance obligatorio del Ejercicio 1**, sin exigir las funcionalidades opcionales excluidas.

Presta especial atención a posibles divergencias en:

- los cinco callbacks requeridos por Enterprise Architect;
- registro/configuración COM y target x64;
- validación estricta de selección EA.Package;
- carga exclusivamente de elementos directos;
- modelo local COM-free y ausencia de persistencia antes de Guardar;
- editabilidad de Name, Alias y Notes;
- Type y Stereotype read-only;
- Notes multilínea;
- comportamiento modal;
- Cancelar, Escape y X sin persistencia;
- commit de la edición activa antes de Save;
- detección de filas realmente modificadas;
- recuperación temporal del elemento mediante ElementId;
- Element.Update() únicamente dentro del flujo explícito de guardado;
- inspección del retorno booleano de Update();
- manejo independiente de Update() == false, excepciones y elementos locked/unwritable;
- continuación del procesamiento ante fallo de una fila;
- AcceptChanges() únicamente después de persistencia exitosa;
- conservación como pending de las filas fallidas;
- Save repetido sin cambios sin nuevas escrituras;
- mensajes claros en español;
- .sln clásica y build Debug|x64;
- README suficiente para ejecución sin asistencia;
- AI Usage Log conforme a los campos exigidos por la consigna;
- evidencia real de selección → grilla → edición → guardado → cambio reflejado en EA;
- ausencia de funcionalidades opcionales dentro del cambio base.

Limitación de validación conocida

Las pruebas manuales en Enterprise Architect confirmaron los escenarios reproducibles del flujo funcional.

No fue posible reproducir de forma segura en el repositorio disponible:

- un elemento realmente bloqueado/no escribible;

- un Element.Update() == false real;
- una excepción COM real durante Save.

Estos caminos poseen implementación y cobertura estática, pero no evidencia runtime real.

No consideres automáticamente esta limitación como éxito ni como fallo.

Inspecciona el código correspondiente y determina si:

1. existe un defecto objetivo que impida cumplir la consigna;
2. la implementación es correcta pero permanece un riesgo residual por falta de reproducción runtime;
3. hace falta una remediación concreta antes de aceptar el cambio.

Distingue explícitamente entre **defecto de implementación** y **limitación de evidencia**.

Ejecuta las verificaciones técnicas razonables disponibles, incluido el build Debug|x64, y revisa el diff/código necesario.

Respetar el workflow nativo de Verify y cualquier protocolo sdd-attempt requerido por Gentle-AI. No eludas gates ni fabriques evidencia.

No modifiques código ni documentación durante esta fase. Si encuentras problemas, repórtalos con severidad, evidencia concreta, archivo/línea cuando corresponda y remediación recomendada.

Prioriza completitud y precisión sobre brevedad artificial.

Al finalizar entrega claramente:

- resultado global de Verify;
- requisitos comprobados;
- issues/blockers encontrados;
- warnings o riesgos residuales;
- estado de la limitación locked / Update()==false / excepción;
- resultado del build;
- cumplimiento de entregables;
- recomendación explícita: pass, remediation required o equivalente nativo.

Persiste el verify-report y los artefactos nativos de revisión correspondientes.

Detente al finalizar sdd-verify. No avances automáticamente a sdd-archive y no implementes correcciones.

Evidencia:

DES-003:

